

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour

Total Marks:30

Annexure A

Dry Run Evaluation

Code of Conduct:

*I **Thamaraiselvi S (192511042)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Thamaraiselvi S

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph: A



Task: Starting from node A, perform a **Breadth-First Search (BFS)**. Complete the following table.

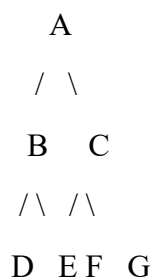
Iteration Queue Visited Node

1
2
3
4
5
6
7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from A.

Complete the table.

3. Initial State :

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1
2
3
4
5

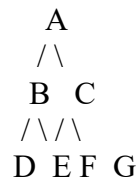
Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

1) Given graph:



CODE:

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

def bfs(start):
    queue = deque([start])
    visited = []

    while queue:
        node = queue.popleft()

        if node not in visited:
            visited.append(node)
            queue.extend(graph[node])

    return visited

print("BFS Traversal:", bfs('A'))
```

OUTPUT:

```
>>>
BFS Traversal: ['A', 'B', 'C', 'D', 'E', 'F', 'G']
>>> ===== RESTART =====
>>>
```

BFS Traversal Order
A → B → C → D → E → F → G
Queue Dry Run

Iteration	Queue Before	Visited Node	Queue After
1	A	A	B, C
2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G
5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	Empty

2) DEPTH-FIRST SEARCH(DFS):

DFS Traversal Order

A → B → D → E → C → F → G

DFS Dry Run

Step Current Node Visited

1	A	A
2	B	A, B
3	D	A, B, D
4	E	A, B, D, E
5	C	A, B, D, E, C
6	F	A, B, D, E, C, F
7	G	A, B, D, E, C, F, G

CODE:

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

def dfs(node, visited):
    visited.append(node)

    for next_node in graph[node]:
        if next_node not in visited:
            dfs(next_node, visited)
```

```
visited = []

dfs('A', visited)

print("DFS Traversal:", visited)
```

OUTPUT:

```
>>>
DFS Traversal: ['A', 'B', 'D', 'E', 'C', 'F', 'G']
>>>
```

3) 8 PUZZLE BFS

Initial state:

```
1 3 6
5 0 2
4 7 8
```

Goal state:

```
1 2 3
4 5 6
7 8 0
```

Solution path:

```
1 3 6  1 3 6  1 3 0
5 0 2 → 5 2 0 → 5 2 6
4 7 8  4 7 8  4 7 8
```

```
→ 1 0 3 → 1 2 3 → 1 2 3
  5 2 6  5 0 6  0 5 6
  4 7 8  4 7 8  4 7 8
```

```
→ 1 2 3 → 1 2 3 → 1 2 3
  4 5 6  4 5 6  4 5 6
  0 7 8  7 0 8  7 8 0
```

Explanation:

- ☐ Expanded states: Initial state → intermediate states → Goal state.
- ☐ States generated: Depends on the order used to generate moves.
- ☐ Solution length: 8 moves.
- ☐ BFS finds the shortest solution because it explores states level by level, so the first goal reached has the minimum number of moves.

CODE:

```
from collections import deque

start = (1, 3, 6,
        5, 0, 2,
        4, 7, 8)

goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

def moves(state):
    result = []
    zero = state.index(0)
    row, col = divmod(zero, 3)

    directions = [(-1,0), (1,0), (0,-1), (0,1)]

    for dr, dc in directions:
        r, c = row + dr, col + dc

        if 0 <= r < 3 and 0 <= c < 3:
            pos = r * 3 + c
            new_state = list(state)

            new_state[zero], new_state[pos] = \
                new_state[pos], new_state[zero]

            result.append(tuple(new_state))

    return result

def bfs(start, goal):
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path

        for new_state in moves(state):
            if new_state not in visited:
                visited.add(new_state)
                queue.append((new_state, path + [new_state]))
```

```
path = bfs(start, goal)

print("Solution Path:")

for state in path:
    print()
    print(state[0:3])
    print(state[3:6])
    print(state[6:9])
```

OUTPUT:

Solution Path:

(1, 3, 6)
(5, 0, 2)
(4, 7, 8)

(1, 3, 6)
(5, 2, 0)
(4, 7, 8)

(1, 3, 0)
(5, 2, 6)
(4, 7, 8)

(1, 0, 3)
(5, 2, 6)
(4, 7, 8)

(1, 2, 3)
(5, 0, 6)
(4, 7, 8)

(1, 2, 3)
(0, 5, 6)
(4, 7, 8)

(1, 2, 3)
(4, 5, 6)
(0, 7, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

|